Dynamic Analysis - Fuzzing

We acknowledge funding provided by Microelectronic Commons and the support of OUSW(R&E).

Other fuzzing directions

- Edge/Cloud mimicking device.
- Edge/Cloud mimicking controller.
- Controller mimicking edge/cloud/device.

Edge/Cloud

- Similar to standard API (MQTT, REST, etc) fuzzing.
- Custom Invalid State Detection.

Invalid States at Cloud/Edge

- Authentication bypass.
  - Login as root.
- Rule violation.
  - Identity and Access management (IAM) policy violation.
- Delays.
  - Denial of service.

Fuzzing Strategies for Edge/Cloud

General fuzzing => Implementation errors:

- Uses generic API/protocol fuzzers, e.g., RESTler.
- Targets crashes, parser bugs, memory-safety issues, malformed-input handling, and API misuse.
- Design-agnostic: does not require understanding the intended IoT application semantics.

Fuzzing Strategies for Edge/Cloud

General fuzzing => Implementation errors:

Fuzzing Strategies for Edge/Cloud

General fuzzing => Implementation errors:

+ Generic, scalable, easy to automate
+ Little domain knowledge required
+ Good for implementation bugs
− Misses many logic/design errors
− Limited semantic understanding

Fuzzing Strategies for Edge/Cloud

General fuzzing => Implementation errors:

ID	Project	Version	Bug Description	Potential Security Issue	0 Day	Status
M1	EMQX	5.6.0	Segmentation Fault in the function dump_module_literals	Denial of Service	×	Confirmed & Fixed
M2	Mosquitto	2.0.18	Double free in handling `PUBLISH` message from brokers	Denial of Service	✓	CVE-2024-3935
M3	FlashMQ	1.12.1	Crash in handling will `PUBLISH` message	Denial of Service	✓	Confirmed & Fixed
M4	FlashMQ	1.14.0	Assertion in saving retain `PUBLISH` message to the database	Denial of Service	✓	CVE-2024-42645
M5	FlashMQ	1.14.0	Assertion in forwarding will `PUBLISH` message	Denial of Service	✓	CVE-2024-42644
M6	NanoMQ	0.17.5	Segmentation Fault in handling `PUBLISH` message	Denial of Service	✓	CVE-2024-42650
M7	NanoMQ	0.17.5	Segmentation Fault in handling `CONNECT` message	Denial of Service	✓	CVE-2023-34488
M8	NanoMQ	0.17.5	Heap buffer overflow in handling `SUBSCRIBE` message	Remote code execution	✓	CVE-requested
M9	NanoMQ	0.17.9	Heap use after free in handling `SUBSCRIBE` message	Remote code execution	✓	CVE-2024-42651
M10	NanoMQ	0.17.9	Segmentation Fault in handling `PUBLISH` message	Denial of Service	✓	CVE-requested
M11	NanoMQ	0.21.10	Socket file description exhaustion caused by session keeping feature	Denial of Service	✓	Confirmed
M12	NanoMQ	0.21.10	Segmentation Fault in handling client requests	Denial of Service	✓	CVE-2024-42646
M13	NanoMQ	0.21.10	Stack buffer overflow in handling `PUBLISH` message	Remote code execution	✓	CVE-requested
M14	NanoMQ	0.22.1	Stack buffer overflow in handling `PUBLISH` message	Remote code execution	✓	CVE-2024-42647
M15	NanoMQ	0.22.1	Memory Leak after receiving message	Memory leakage	✓	CVE-2024-42649
M16	NanoMQ	0.22.1	Heap buffer overflow in handling `CONNECT` message	Remote code execution	✓	CVE-2024-42648
M17	NanoMQ	0.22.4	Heap use after free in handling `PUBLISH` message	Remote code execution	✓	CVE-requested
M18	NanoMQ	0.22.4	Memory leak in receiving message	Memory leakage	✓	CVE-requested
M19	NanoMQ	0.22.4	Segmentation fault in message	Denial of Service	✓	CVE-requested
M20	NanoMQ	0.22.4	Memcpy-param-overlap in handling `PUBLISH` message	Memory Corruption	✓	CVE-requested

Table 4: Memory bugs discovered in MQTT brokers by MBFuzzer.

Fuzzing Strategies for Edge/Cloud

Semantic fuzzing => Design or logic errors:

- Uses custom, application-specific fuzzers.
- First reconstructs or infers the cloud-side specification, state machine, or device-cloud semantics.
- Generates inputs and event sequences that violate expected invariants.
- Targets authorization flaws, inconsistent device state, unsafe automation, and other logic bugs.

Fuzzing Strategies for Edge/Cloud

Semantic fuzzing => Design or logic errors:

Fuzzing Strategies for Edge/Cloud

Semantic fuzzing => Design or logic errors:

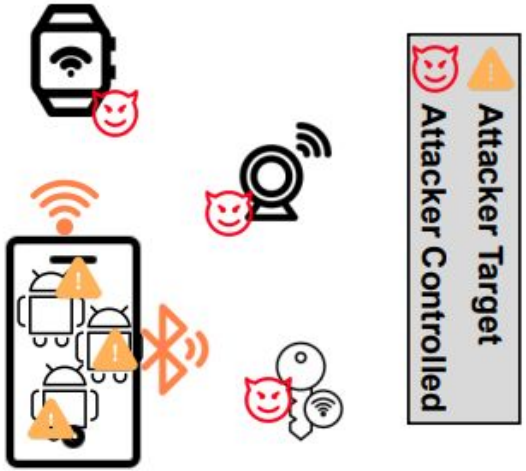
+ Finds deep logic and design flaws
+ Detects “system works, but behaves incorrectly”
+ Can test multi-step workflows and invariants
− Requires system-specific knowledge/modeling
− Higher development and maintenance cost

A researcher using the handle **tokay0** took apart a Shark RV2320EDUS robot vacuum and found that its embedded AWS IoT certificate is allowed to publish and subscribe to topics for any Shark device in the same AWS Region, not just itself.

Fuzzing Controller, i.e., Android App

- Android App Testing:
  - From user side: Mimicking bad user.
    - Extensive works.
  - From device side: Mimicking bad IoT device
    - No work: Nice Master thesis!
    - Closest work: RearFind (SAC 2025) -- static analysis.

Fuzzing Controller from Device




```
1 public void onReceive(Context context, Intent intent) {
2     WifiManager wm = (WifiManager) context.getSystemService("wifi");
3     if (intent.getAction().equals("android.net.wifi.SCAN_RESULTS")) {
4         BAB.this.apList = wm.getScanResults();
5         ...
6         List<String> result = new ArrayList<>();
7         for (int i = 0; i < BAB.this.apList.size(); i++) {
8           ScanResult scanResult =  (ScanResult) BAB.this.apList.get(i);
9           ...
10          result.add(scanResult.SSID);
11        }
12        BAB.this.callWebInterface("setSsid", BAB.this.gson.toJson(result));
13        ...
```

Security Analysis Standards and Certifications

- Standards:
  - Recommendations for best practices.

- Certifications:
  - Verified to follow certain practices.

Static Analysis (SAST) Standards

- IEC (international electrotechnical commission) 62443-4-1 → secure-development/process requirement.
- NIST IR 8397 / SSDF → explicitly recommend code analysis.
- ISO/IEC TS 17961 → statically checkable C security rules.
- MISRA C/C++ → embedded coding rules.
- CERT C/C++ → security-oriented coding rules.

SAST Standards

- Why do we need?
- Which standards to follow?

SAST Standards

- Why do we need?
  - Consistency, baseline, and measurability.

- Which standards to follow?
  - Depends on organization.
  - Depends on your development practices.
  - Use language specific checkers to prevent implementation errors.

SAST Certifications

- ISASecure SDLA (https://isasecure.org/) :
  - Certifies the secure development lifecycle, Based on IEC 62443-4-1.
- ISASecure ICSA / CSA:
  - Certifies industrial IoT devices, gateways, and components. Combines lifecycle review, security testing, and vulnerability assessment.
- UL 2900 / UL CAP :
  - Cybersecurity certification for network-connected products and medical devices.

SAST Certifications

They do not certify specific SAST tool but a broader use of SAST in software development.

Why we need certifications?

Security assurance, Government use.

Dynamic Analysis: Standards

- NIST IR 8397 — Explicitly recommends fuzzing and dynamic testing
- ETSI EN 303 645 / TS 103 701 — Unexpected-input validation for consumer IoT
- ETSI TR 104 287 — IoT security validation including firmware fuzzing
- IEC 62443 / ISASecure — Protocol robustness and malformed-message testing
  UL 2900 — Malformed-input, fuzz, and penetration testing

Dynamic Analysis Certifications

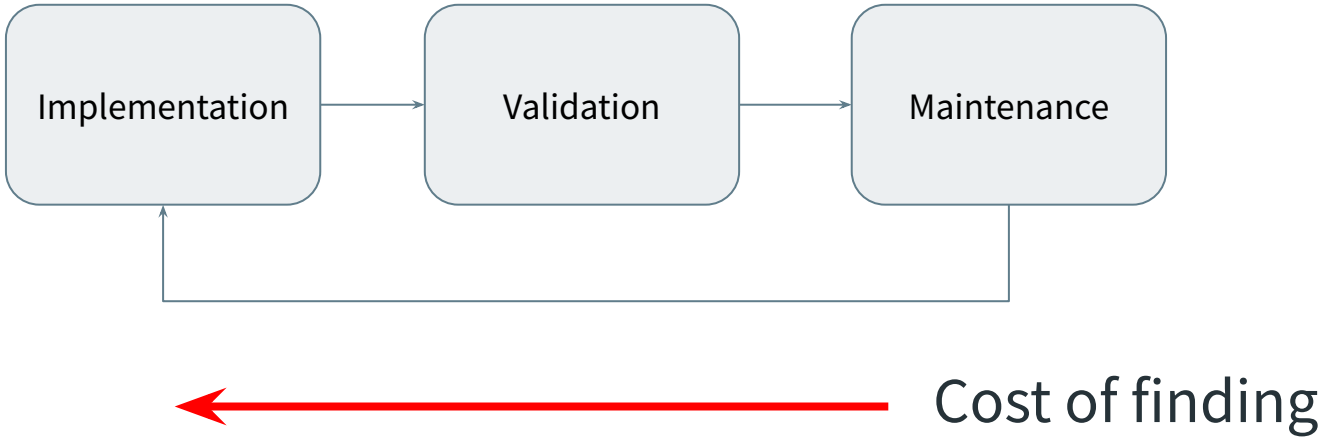
- No specific certification.

- Used as evidence within a broader product-security certification.
  - ISASecure CSA / ICSA reviews evidence that the supplier performed robustness testing

Where to use security analyses?



Where to use security analyses?



Where to use static analyses?



Where to use static analyses?

- Implementation: For coding errors and best practices.
  - E.g., Check for simple patterns, compiler checkers.
- Validation: For flow based bugs and deeper analysis.
  - E.g., Custom CodeQL queries.
- Maintenance: Regressions.
  - E.g., Scan only modified files.

How to use static analysis

- As part of IDE (Simple checker).
- Overnight runs (Deeper and complex checks).
- CI/CD (Sanity checks, simple checks and optional deeper checks):
  - GitHub Worklows.

Where to use dynamic analyses (i.e., Fuzzing)?



Where to use dynamic analyses (i.e., Fuzzing)?





Where to use dynamic analyses (i.e., Fuzzing)?

- Implementation: Function level fuzzing.
- Validation: System level fuzzing.
- Maintenance: Directed fuzzing
  - Focus only on the modified code.